



Author

Pranjal Sinha, IIT Kharagpur

Mentor

Prof. Jagadeesh Kadiyam, IIT Kharagpur

Comprehensive Simulation Setup Document

Preface

This document provides a comprehensive, step-by-step guide to setting up a simulation environment for studying Autonomous Surface Vessels (ASVs). It is designed for users with a Windows 10 or 11 system who want to build a dedicated workspace from scratch. The guide covers everything from the initial Linux installation to running code files, executing test commands, and exploring future development opportunities. Because a native Linux environment offers the best build freedom and community support, this documentation focuses exclusively on configuring a physical dual-boot system.

The Linux Operating System

First, is the OS that will be used for the setup. It is preferable to use a linux operating system due to its freedom of installation, build workspace and open source community. Windows and MacOS can be used as well, but we won't be considering them in this document. An alternative to installing the core linux OS can be to use WSL's or VM's. But again, VM's are not preferred. Whereas, the process for setup on WSL may be available as a future improvement to this documentation.

Here, I am assuming that you have Windows 10 or 11 OS in your pc. (I do not know how to do it for a Mac, you can research about it and publish a process for the same).

Let's first clarify which distribution of Linux I am recommending you to install and use. My recommendation is to use Ubuntu-22.04 LTS Jammy Jellyfish.

Install the LTS (Long Term Support) version of the OS distribution.

Why only Ubuntu-22.04 LTS? The gazebo version, ROS version and versions of other dependencies and libraries are only compatible with the 22.04 distro. The setup will fail or produce massive errors when performing it in a different OS.

Please note that Ubuntu-22.04 is an older version in the timeline and can have issues with installation of wireless connection drivers, graphics drivers and more. These issues are non-trivial and dealing with them is a system specific task. It requires research into community forums with available fixes, help from a mentor who has done it before, or trustworthy AI searches.

An advice, try installing Ubuntu-22.04 first. If it is broken and the internal issues are not resolvable, try using the WSL (Windows Subsystem for Linux) instead.

Partitioning The Drive (In Windows)

It is highly likely that your Windows system already has a lot of files that you cannot back up or destroy. This method involves installing Ubuntu alongside your Windows system. To do this safely, you **need** to allocate at least 90GB of storage space for Ubuntu. This is called **Dual Booting** your pc. If your system does not have the space required. You can install and store the OS in a separate Hard Drive.

To allocate space, you need to partition the existing storage in your system. Right now it is completely allocated to windows, and its bootloader. To partition it, follow these steps:

1. Press Win + R.
2. Type diskmgmt.msc and press enter
3. You will see this: <insert image>.
4. Right click on the D drive partition. (or if it's entirely a C drive then do it on that).
5. Select shrink volume
6. In the "enter the amount of space to shrink in MB"
Write the amount of space you want to allocate to the partition in MB (Mega bytes).
For ~91GB, type this number: 91000.
7. Click shrink.
8. You should see a partition shown as unallocated in your system with ~91GB of storage.

(attach images).

Steps for Installing the Ubuntu OS

Moving on. To Install the OS, you need the following:

1. A bootable image of the OS in a pendrive.

To understand what is a bootable OS image, read these useful pages:

[QNX.com OS Images](#)

[Lenovo - What is a boot device](#)

2. you require **rufus**.
3. Lastly, you require a 16+ GB pendrive to store the bootable image.

1. Downloading the .iso image file

The bootable image is in the form of a **.iso** file. For the ubuntu installation you require the appropriate .iso file for Ubuntu-22.04 LTS. Follow the steps below:

- a. Go to this link <https://releases.ubuntu.com/jammy/>
- b. Download the Desktop Image in the Downloads folder. Screenshot shown below.

Desktop image

The desktop image allows you to try Ubuntu without changing your computer at all, and at your option to install it permanently later. This type of image is what most people will want to use. You will need at least 1024MiB of RAM to install from this image.

64-bit PC (AMD64) desktop image

Choose this if you have a computer based on the AMD64 or EM64T architecture (e.g., Athlon64, Opteron, EM64T Xeon, Core 2). Choose this if you are at all unsure.

Now, you have successfully downloaded the .iso image file for Ubuntu-22.04 LTS.

2. Installing Rufus

The textbook definition of what rufus is:

‘Rufus (The Reliable USB Formatting Utility, with Source) is a free, open-source, and portable application for Microsoft Windows (version 8 and later) used to format and create bootable USB flash drives.’

You will use rufus to create the bootable USB drive with the Ubuntu-22.04 .iso image file.

To install rufus, follow these steps below.

- a. Go this link <https://rufus.ie/en/>
- b. For an x64 bit windows system: Scroll down and download either of these files:

rufus-4.14.exe	Standard	Windows x64	1.9 MB
rufus-4.14p.exe	Portable	Windows x64	1.9 MB

- c. For x86 bit windows system: Scroll down and download this file:

rufus-4.14_x86.exe	Standard	Windows x86	1.9 MB
------------------------------------	----------	-------------	--------

d. For ARM 64 bit windows system: Scroll down and download this file:

rufus-4.14_arm64.exe	Standard	Windows ARM64	5.1 MB
--------------------------------------	----------	---------------	--------

Source of Confusion, what is x64, x86 and arm64? And what does your system have? These are the CPU architectures of your system. We won't go into the details. But, programs are made in a manner so that they are compatible with these architectures. To figure out your system CPU architecture, follow these steps:

- I. Press Win + R
- II. Type msinfo and press enter
- III. Under 'System Summary' look for 'System Type'
- IV. You will see the displayed CPU architecture of your system

You should have successfully installed rufus.

3. Making the bootable USB drive

Making a bootable USB drive involves formatting the drive. Make sure you have backed up all the content in the USB drive.

Startup rufus (double click the .exe file you have downloaded for rufus).

Under Device Properties→Device, select your USB drive.

Under Device Properties→Boot selection, select the ubuntu-22.04 .iso file we have downloaded

Do not change any other selections.

Click Start. It may prompt you with 'This will format your USB device'. Press yes/ok.

Now wait for the green 'Status' bar to get filled. Once done. Close rufus. Your USB drive name may have changed to UBUNTU or similar.

You have successfully made a bootable image USB Drive for Ubuntu-22.04 LTS.

The Detailed Process of Installation requires accessing the system BIOS, changing a few settings, and then booting up the USB drive. This will be included as a future improvement to the document. In the meantime please look at the community forums, or youtube for this part of the process.

Post Installation Instructions

After successfully installed and booted to the Ubuntu-22.04 LTS OS, you can set up the UI appearance to your convenience as long as it does not break the system.

NOTE: do not use **sudo (super user do)** commands in the terminal unknowingly of what affects it has. sudo means root (admin) access to the system internal files where adding and changing them can make or break your OS and your work.

In Ubuntu, or any other linux operating system, make the terminal your friend. It is the main aspect of a linux system. Any and all work in linux can be done through the terminal with high efficiency. Ubuntu is a more user friendly version of linux with a productive community. You should be able to bravely and confidently use the terminal for any purpose. With use and time, you will be able to.

Before starting off, let's make a directory where you can **copy** your workspace and test custom built packages and files.

```
cd ~/
mkdir test_gz_ws
```

This workspace is for your own personal use. You can choose not to follow it.

One more thing to keep in mind. I use **code** in the syntax here to open files. This only works if you have VisualStudioCode installed in your system. So I advise you to do so. Otherwise, if you wish to use any other IDE, use **nano**, or search for alternatives.

So with the mind straining dual boot completed. Let's move onto all the requirements for the Gazebo Wave Simulation + Ardupilot SITL setup.

Requirements for the Simulation Setup

Requirements:

1. Programming Languages:
C++ Standard, Python3
2. Git
Git is a tool used to clone, append, make commits locally and perform the final push to the main repo.
3. ROS2 Humble Hawksbill
Provides the file architecture, colcon build tool and creation of launch files.
4. Gazebo 7.1.0 or higher.
This setup uses Gazebo Harmonic → gz sim 8.12.0 Harmonic

5. Ardupilot and Ardupilot Gazebo.
SITL for the simulation/
6. MAVLink Proxy
Communication between the vehicle and ground station.
7. GitHub repositories.
GitHub repos with the content made available for such simulation setups.

Progressive Compliance of Requirements

1. Installing the Programming Languages

The required programming languages are Cpp (or C++, same thing) and Python 3.

To use these languages for coding and running, we need the languages themselves and their compilers.

First we will set up C++ and the g++ compiler.

C++ already has an installed version on your system by default. You can check it by typing this command in the terminal:

```
cpp --version
```

For the g++ compiler, it does not exist by default. To install it run this command:

```
sudo apt install build-essential
```

build-essential is a meta package in ubuntu that installs a suite of tools required for compiling software, particularly C and C++.

To verify the installation and check the version, run this command:

```
g++ --version
```

Moving onto installing Python. You will use the python3 version as it is the latest version.

Python is not installed on the system by default. So you have to install it manually.

But you can confirm it with:

```
python3 --version
```

If nothing shows up, python3 is not installed.

You can do that by following the commands below: (copy and paste in the terminal)

```
sudo apt install python3-pip
```

```
sudo apt install python3-venv
```

python3 is a pip installation. pip means (Pip Installs Packages). But let's not delve into those details.

python3-venv means virtual environment area which is useful when installing or running certain packages. We won't be needing it in the scope of this documentation. But, it is useful to have for later use.

Final Check, run these commands: If you get a version output, then your installations were successful.

```
cpp --version
```

```
g++ --version
python3 --version
```

2. GIT Installation and use case

Git is a tool used to clone, append and push github repos. It is helpful in making local changes to the repository and then pushing the final changes to the main repository. It also stores past versions of the repo in case we need to revert things or compare changes.

Here, we will only be using Git for safely cloning the repository we want to use. However, Git is useful for any solo or team building of code for any project.

To install git, just type this command in the terminal:

```
sudo apt update
sudo apt install git
```

Verify the installation and git version with:

```
git --version
```

Installing Git by itself is pretty useless. You need two more things: a GitHub account and an SSH key that is connected to your account.

Setting up your GitHub account is simple, just go to github.com and make one.

Setting up the SSH connection requires more attention. In fact I will just add the link to the site that already has the comprehensive step by step explanation for setting up an SSH connection with your GitHub account.

[Connect with SSH](#)

It's quite easy if you follow the steps properly.

Also keep in note the password you use for the SSH key as Git and SSH may prompt you to enter it for permissions.

Note: Do not use VPN's as SSH uses network IP protocols to establish a connection. VPN's change those from the user side and thus no connection is established. Git and SSH won't work with VPN's ON.

Well there are workarounds but are complex to understand and work out. So just turn them OFF.

You can verify that Git has been setup and is communicated well by typing this in the terminal:

```
ssh -T git@github.com
```

You should receive an output like:

```
Hi <username>! You've successfully authenticated, but GitHub does not provide shell
access.
```

With all of this done, you should be able to use Git. We will use it to clone the relevant repo's mentioned in the lower sections.

3. ROS2 Installation

ROS2 Humble Hawksbill is a LTS version of ROS compatible with Ubuntu 22.04. You will use ROS2 Humble and its syntax to build all packages and run/launch them.

ROS2 provides the build tools like colcon essential to build all the packages that will be used to build the simulation setup. In the future, you will also build packages for the navigation stack, control and other proprietary software for your ASV.

To install ROS2 humble, you can have a look at the official documentation of ROS which provides a detailed step by step installation guide. Alongside that, it offers beginner, intermediate and advanced guides on how to use ROS2 for your work. The link is provided below,

[ROS2 Humble](#)
[ROS2 Humble Ubuntu Installation](#)
[ROS2 Humble Tutorials](#)
[ROS2 Humble Concepts](#)

These 4 pages and their associated tabs are the most crucial part of understanding ROS2 humble. I advise you to learn about how to work with ROS2 before making custom changes to the workspace.

4. Gazebo Installation and Version Control

I would prefer you to use the gazebo version similar to what this document uses.

gz sim 8.xy.z harmonic

Don't worry about the specificity of the version besides gz sim 8.

Multiple Gazebo versions on your system can clash and cause errors. To resolve this beforehand, we shall remove all versions of gazebo that may exist on your system.

In the terminal:

```
dpkg -l | grep -i gazebo
dpkg -l | grep -i ignition
dpkg -l | grep -i gz-

sudo apt remove --purge '*.gazebo.*'
sudo apt autoremove

sudo apt remove --purge '*.ignition.*'

sudo apt remove --purge 'gz-*
```



```
sudo apt autoremove
```

```
sudo apt remove --purge ros-humble-gazebo-ros-pkgs ros-humble-gazebo-ros  
sudo apt autoremove
```

This ensures that there is no gazebo version on your system. Being on the safer side, you can clear the system cache as well. In the terminal:

```
sudo apt clean
```

Now we know that there is no gazebo software on your system. We can now install the gazebo version gz sim 8.xy.z harmonic. In the terminal:

```
sudo apt-get install gz-harmonic
```

Verify the gazebo version with:

```
gz sim --version
```

It should show “Gazebo Sim, version 8.xy.z”

So we have completed the Gazebo simulation software installation. It provides the UI where all the graphics, spawn and motion takes place.

5. Ardupilot Installation

ArduPilot is an open source software suite with controls for various unmanned vehicles. You will be using ArduPilot for the controls.

Amongst the software suite is the SITL framework (Simulation in the Loop). This framework allows the firmware to run as a native process on the desktop without requiring the hardware, like pixhawk.

To Install Ardupilot, follow these steps in the terminal:

- a. Clone the ArduPilot repo:

```
git clone --recurse-submodules https://github.com/ArduPilot/ardupilot.git
```

- b. Open the cloned repo directory:

```
cd ardupilot
```

- c. Run the official dependency installer. Installs all the required dependencies for the ArduPilot to run.

```
Tools/environment_install/install-prereqs-ubuntu.sh -y
```

- d. Reload your profile. This adds the sim_vehicle.py to your path. This is used to selectively run a vehicle control and SITL from the suite.

```
./profile
```

- e. Run an example to see whether Ardupilot builds the vehicle controls and SITL.

```
cd ~/ardupilot/ArduCopter  
sim_vehicle.py -w
```

The output should be a list of different files being compiled and built for the vehicle control. If the compile is completed without errors and the control runs

completely, you can say the Ardupilot installation was successful.

After this, we have to install the Ardupilot Gazebo Plugin so that it can communicate with the gazebo world and use actuators and sensors defined in the .sdf file of the gazebo world being used. To install it, follow these steps in the terminal:

- a. First, install all the required dependencies:

```
sudo apt update
sudo apt install libgz-sim8-dev rapisjson-dev
sudo apt install libopencv-dev gstreamer1.0-dev \
libgstreamer-plugins-base1.0-dev gstreamer1.0-plugins-bad \
gstreamer1.0-libav gstreamer1.0-gl
```

- b. The plugin package is part of the github repo that you will be cloning for the simulation setup. The rest of the setup is part of building the package.

6. MAVLink Proxy Installation

MAVProxy is a command line, console based ground control station (GCS). MAVLink Proxy is the sub component designed for the ArduPilot ecosystem. It functions as a portable tool to control the vessel, monitor telemetry and route messages over various network interfaces. Here we will use the UDP interface with the IP: 127.0.0.1

To install it, in the terminal:

```
pip3 install MAVProxy
```

To verify the install, try:

```
mavproxy.py --version
```

It should output with the MAVProxy version installed.

7. Cloning the GitHub Repositories and building the workspace

Here, you will require the git commands to clone the repo cleanly. If you have done the git setup via ssh keys properly, you won't have any issues at this stage.

Precaution: keep your VPN off during the clone process.

repo link:

In the terminal:

```
cd ~/
git clone git@github.com:StrikePS/Autonomous-Surface-Vessel-ASV-.git
```

A. Ardupilot Gazebo Plugin Build

After cloning the repo. First we will clone and build the ardupilot gazebo plugin. For that, follow the commands below:

```
cd ~/Autonomous-Surface-Vessel-ASV-/gz_ws/src
git clone https://github.com/ArduPilot/ardupilot_gazebo
export GZ_VERSION=harmonic
cd ~/Autonomous-Surface-Vessel-ASV-/gz_ws/src/ardupilot_gazebo
```

```
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=RelWithDebInfo
make -j4
```

You can verify the build was successful by running the following command:

```
find / -name "libArduPilotPlugin.so" 2>dev/null
```

This checks if the libArduPilotPlugin.so file exists or not. If it does then the build was completed well. This is the most important ardupilot gazebo plugin file for the simulation setup.

B. Build the Wave Simulation package

Here we build the packages used for starting the wave simulation world and spawn our boat in the gazebo world.

Credits to 2 github repo's who's appended version for our use case are used in this setup. They are mentioned below.

1. The [asv_wave_sim](#) github repository.
These packages were earlier cloned from the "asv_wave_sim" github repo by "srmainwaring".
It is used to create the gazebo world with water surface, waves, and any included models. It uses the FFT and CGAL libraries to render the world and solve for hydrodynamic physics.
2. The [SITL_Models](#) github repository.
Another repo used is the Ardupilot/SITL_Models, that provided us with a base ground work for the .sdf file of our boat.
It provided the famously used "BlueBoat" model.sdf with appropriate hydrodynamics that can be used to test out custom control packages and navigation stacks. It was later used as a base to create a model.sdf of our custom boat (catamaran).

To build the workspace, you can copy paste all of these lines below in the terminal:

```
source /opt/ros/humble/setup.bash

cd ~/Autonomous-Surface-Vessel-ASV-/gz_ws
export GZ_VERSION=harmonic
colcon build --merge-install --cmake-args \
-DCMAKE_BUILD_TYPE=RelWithDebInfo \
-DBUILD_TESTING=ON \
-DCMAKE_CXX_STANDARD=17 \
--event-handlers console_direct+ \
--executor sequential --parallel-workers 2
source ./install/setup.bash

cd
```

```
~/Autonomous-Surface-Vessel-ASV-/gz_ws/src/asv_wave_sim/gz-waves/src/gui/plugins/waves_  
control  
mkdir build && cd build  
cmake -Wno-dev .. && make
```

The workspace should build without errors. There may be a Cmake warning like this:

CMake Warning:

Manually-specified variables were not used by the project:

CMAKE_CXX_STANDARD

It comes from the command line “-DCMAKE_CXX_STANDARD=17”.

The launch file, “simulation_launch_pkg” is a pure python package, it does not use C++. So the error is expected, and harmless. And ...

CMake Warning at /usr/lib/x86_64-linux-gnu/cmake/CGAL/CGALConfig.cmake:92

(message):

CGAL_DATA_DIR cannot be deduced, set the variable CGAL_DATA_DIR to set the default value of CGAL::data_file_path()

Call Stack (most recent call first):

CMakeLists.txt:240 (find_package)

But do not worry, the warning is harmless and can be ignored. gz-waves in the asv_wave_sim package uses CGAL libraries for ocean surface mesh geometry. But, it never calls the CGAL::data_file_path() which is just a helper function used in CGAL's own example programs. We can move forward if there are no other errors.

C. Run the simulation

After performing the build. It's trivial to run the setup altogether.

- a. Source ros2

```
source /opt/ros/humble/setup.bash
```

- b. Source the workspace (although we did that in the build process).

```
cd ~/Autonomous-Surface-Vessel-ASV-/gz_ws  
source install/setup.bash
```

- c. Use the ros launch file to launch all the packages and nodes.

```
ros2 launch simulation_launch_pkg boat_simulation_launch.py
```

D. Future runs and updates to the workspace

Whenever you make changes to the packages and files in the workspace, always rebuild the workspace using

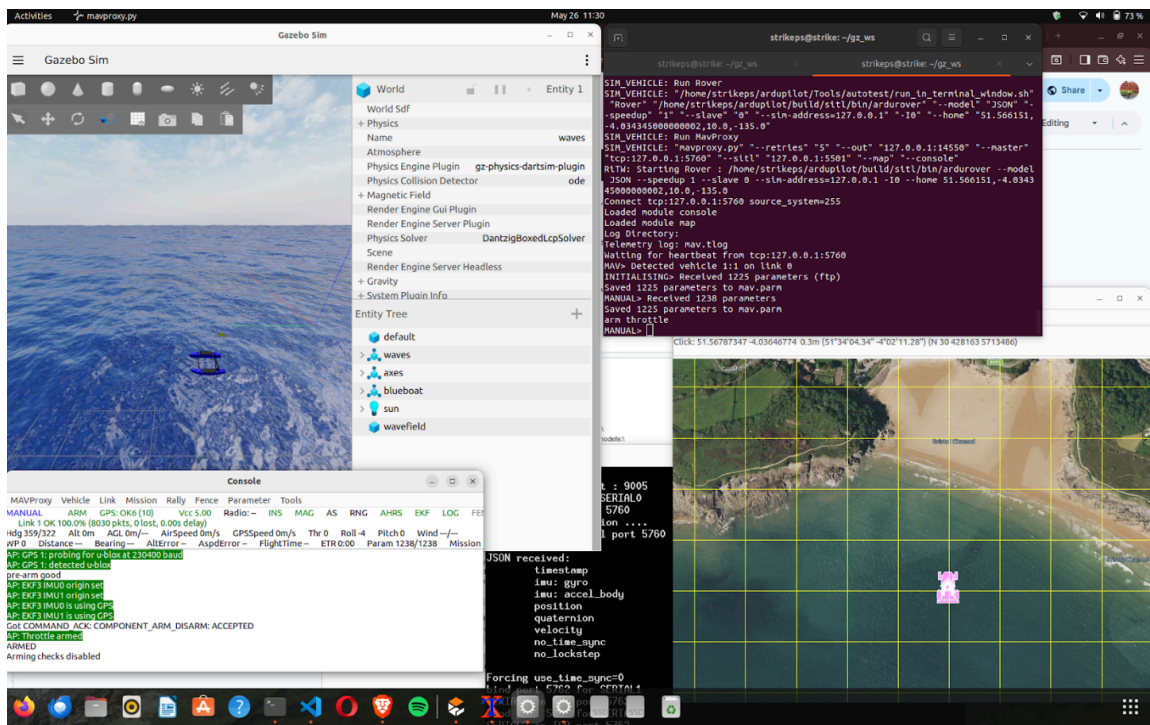
```
colcon build --merge-install
```

This builds the workspace with a merged install directory layout which is easier to navigate when exporting build environments or importing them elsewhere.

After building, always source the workspace again to catch up with the recent build and then run the setup using the launch file:

```
cd ~/Autonomous-Surface-Vessel-ASV-/gz_ws
source install/setup.bash
ros2 launch simulation_launch_pkg boat_simulation_launch.py
```

Below is a screenshot of the windows the launch will spawn.



You can see the Gazebo wave sim, a console map, the mavlink console, terminal window.

Understanding the Workspace

There are key aspects in the workspace that need to be taken into account before running the code. After a series of changes to files themselves or the locations of the files, we need to assure that those changes are reflected in the crucial files involved with the build, compile and run of the workspace.

Let's visit each file one by one.

1. Waves.sdf World file

waves.sdf is located in this directory:

`~/Autonomous-Surface-Vessel-ASV-/gz_ws/src/asv_wave_sim/src/gz-waves-models/worlds/`

The waves.sdf file is what is being compiled by gazebo to form and run the simulation

world. All files that waves.sdf depends on are imported into the .sdf itself. Open the file and have a look.

[gz-sim-physics-system](#) is a plugin that calls FFT to calculate forces, damping, collisions, drift, et cetera.

[gz-sim-scene-broadcaster-system](#) is a plugin that calls CGAL and FFT to render the ocean waves in the simulation.

[gz-sim-user-commands-system](#) calls the input gz services or gz topics to take user commands.

[gz-sim-imu-system](#), [gz-sim-navsat-system](#) call the individual sensors to relay data from the simulation as a gz topic. The sensors are defined in the .sdf of the model you are using.

```
<include>
  <pose>0 0 0 0 0 0</pose>
  <uri>model://waves</uri>
</include>
```

This includes the wave model that will be rendered on the gazebo screen. The physics is again governed by the [gz-sim-physics-system](#) plugin. Any model, whose folder path has been exported in the environment can be included here. It is advisable to make models in the

[~/Autonomous-Surface-Vessel-ASV-gz_ws/src/asv_wave_sim/src/gz-waves-models/world_models](#) directory so that all models made are located in the same space. Also avoid having the same nomenclature as to avoid conflict for the system calls to understand which directory to use. The directory already contains models that can be used.

If you scroll down, you will find lines like these:

```
<include>
  <pose degrees="true">0 0 0 0 0 0</pose>
  <uri> model://blueboat</uri>
</include/>
```

This includes the blueboat model to spawn and simulate according to world physics. The physical properties and mesh of the blue boat are defined in "it's" *model.sdf*, *.dae*, *.stl* files in the blueboat directory, wherever it is, and uses that for appropriate physics via FFT.

In the same directory as waves.sdf, you might see some other world files like waves_wind, which includes blowing wind effects on the ocean world. And more but we need not go into more detail. We are mostly concerned with waves.sdf

Some of you may be interested in changing wave parameters. For that you can explore

the files in this directory:

`~/Autonomous-Surface-Vessel-ASV-/gz_ws/src/asv_wave_sim/src/gz-waves-models/world_models/waves/`

`~/Autonomous-Surface-Vessel-ASV-/gz_ws/src/asv_wave_sim/src/gz-waves-models/world_models/regular_waves/`

`~/Autonomous-Surface-Vessel-ASV-/gz_ws/src/asv_wave_sim/src/gz-waves-models/world_models/ocean_waves/`

Furthermore, anyone interested in diving deep, please consider exploring all files in the `asv_wave_sim` package to smartly add or append files for render and physics. An improvement that can be considered is: Adding wave making render and physics and wave interaction for accurate hydrodynamic response in the simulation.

2. BlueBoat model.sdf

Navigate to this directory:

`~/Autonomous-Surface-Vessel-ASV-/SITL_Models/Gazebo/models/blueboat/`

This directory contains all the specifications required to spawn a blueboat in the world and have it simulate with appropriate physics.

You can look for lines like `<mass> <inertia> <!-- Hydrodynamics plugin --> <!-- motor thruster ccw -->`. These are responsible for taking care of the physics used during simulation.

For the sensors, there is `<link name="imu_link">` and `<link name="gps_link">` for now. They attach these imu and gps sensors to the boat at a location given by `<pose>` relative to the blueboat.

Notice lines like `<noise type> <stddev> <bias_mean>` et cetera to simulate the sensor noise and inaccuracy of the real world. Here we have used **gaussian** noise type.

3. Custom Catamaran model.sdf

A custom Catamaran model, built at the department of Ocean Engineering and Naval Architecture at IIT Kharagpur was measured and replicated in CAD software. It was built using Fusion360, exported as a .stl file and converted to a .dae file via Blender-3.0. Later the model.sdf file was made using the BlueBoat model.sdf as the base.

4. Custom ROZ_GZ_BRIDGE

There is a custom built ROS_GZ_BRIDGE available in this workspace.

`code`

`~/Autonomous-Surface-Vessel-ASV-/gz_ws/src/wave_sim_bridgeup/src/custom_bridge.cpp`

This takes care of any issue of version mismatch when installing the bridge via the normal installation process. This file contains a lot of complicated sections and will not be covered by this documentation.

5. Start Gazebo Simulation and ArduPilot SITL .sh Script

Navigate to this file:

```
code
~/Autonomous-Surface-Vessel-ASV-/gz_ws/src/simulation_launch_pkg/scripts/start_gazebo_wave_sim.sh
```

This file contains the script to startup the gazebo gui and simulation.

As you can see, there are a lot of exports here, like `GZ_SYSTEM_PLUGIN_PATH`, `GZ_GUI_PLUGIN_PATH`, et cetera.

For you, the most important section are these:

```
cd "$1/Autonomous-Surface-Vessel-ASV-/gz_ws" || exit 1
```

Keep the correct file path to the workspace you are using.

```
export GZ_VERSION=harmonic
```

```
export GZ_IP=127.0.0.1
```

This exports the environment to gazebo sim harmonic and avoids gazebo version error.

The IP is used for the communication protocol between ArdupilotSITL (connected to gazebo) and MAVProxy. They require the same IP to transfer data to and fro, like commands or sensor data.

```
export
```

```
GZ_SIM_RESOURCE_PATH="$1/Autonomous-Surface-Vessel-ASV-/gz_ws/src/asv_wave_sim/gz-waves-models/models:\
```

```
$1/Autonomous-Surface-Vessel-ASV-/gz_ws/src/asv_wave_sim/gz-waves-models/world_models:\
```

```
$1/Autonomous-Surface-Vessel-ASV-/gz_ws/src/asv_wave_sim/gz-waves-models/worlds:\
```

```
$1/Autonomous-Surface-Vessel-ASV-/SITL_Models/Gazebo/models:\
```

```
$GZ_SIM_RESOURCE_PATH"
```

These are exports for models included in the **waves.sdf** file. Without these exports, waves.sdf will not know where the blueboat directory, or waves world model directory exists, to extract their data, compile it together and simulate it.

Make sure that these path exports are consistent with the directories where you have saved your models.

6. Start MAVProxy with .sh scripts

Navigate to this file:

```
code ~/Autonomous-Surface-Vessel-ASV-/gz_ws/src/simulation_launch_pkg/scripts/start_sitl.sh
```

This file contains the start script for mavrpxy. And also the script to input the spherical coordinates of the boat into the gazebo world.

```
gnome-terminal -- bash -c '
```

This opens a new terminal to run the script for MAVProxy.


```
sim_vehicle.py -v Rover -f rover-skid --model JSON --console --map \
  --custom-location=$1,$2,$3,$4
exec bash
'_ "$LAT" "$LON" "$ALT" "$HDG"
```

This starts the MAVProxy with relevant consoles like control console, map, et cetera at the specified spherical coordinates (latitude, longitude, altitude).

You can delete the `--map` section from the script and manually open a ground station software like QGC.

```
gz service -s /world/waves/set_spherical_coordinates \ .....
```

After MAVProxy has started, the spherical coordinates are inputted to the gazebo world so that gps data is aligned with the map. Otherwise we will receive erroneous gps data that does not match our console map or ground station map.

7. Launch file

Navigate to this file:

```
code
~/Autonomous-Surface-Vessel-ASV-/gz_ws/src/simulation_launch_pkg/launch/boat_simulation_launch.py
```

This file contains the launch description to run scripts and start ros nodes in order. This launch file starts all the necessary files to run the basic simulation setup, whose image has been displayed above. You can update this launch description or make your own to run all relevant scripts and files at once instead of doing it manually in multiple terminal windows.

It also is responsible to set the spherical coordinates of where to spawn the boat.

```
lat_arg = DeclareLaunchArgument('lat', default_value='22.317677')
lon_arg = DeclareLaunchArgument('lon', default_value='87.301955')
alt_arg = DeclareLaunchArgument('alt', default_value='0.0')
hdg_arg = DeclareLaunchArgument('hdg', default_value='0')
```

You can change the coordinates to your boat's location.

8. ASV localization

A well made EKF has already been implemented here.

It returns the **relative cartesian pose** from the initial spawn pose.

You can look at the EKF built here:

```
code
Autonomous-Surface-Vessel-ASV-/gz_ws/src/asv_localization/asv_localization/GzSim_StateEstimate.py
```

Explanation of this code will not be done in this documentation.

You can look at the output of the EKF using this command in a “new terminal window”:

```
cd ~/Autonomous-Surface-Vessel-ASV-/gz_ws
source install/setup.bash
ros2 run asv_localization echo_state_estimate
```

Spawning the Catamaran Model

A model of the catamaran is available at the department of Ocean Engineering and Naval Architecture at IIT Kharagpur. We have measured and created a CAD model of the catamaran that can be used for simulation and testing in various aspects of robotics, hydrodynamics, CAD buildup and more.

The specs of the model are not mentioned in this document. The CAD models of the bare hull and joined hull is available and can be found in the repo at this location:

```
cd ~/Autonomous-Surface-Vessel-ASV-/ASV_CAD/Catamaran_v1/
```

In the cloned repo, navigate to this folder:

```
cd ~/Autonomous-Surface-Vessel-ASV-/Custom_Models/Gazebo/models/newmodel
```

The newmodel directory holds the .sdf and .stl files of the catamaran.

The model.sdf file in the directory is used to spawn the catamaran with the hydrodynamics, and SITL plugins. This way the model behaviour is similar to real world behaviour and other control algorithms can be built on top of the SITL.